1 Simple Lightcurve Forward Model

In the notation of Kaasalainen, [1], the light curve contribution of an asteroid surface patch $\mathrm{d}s$ that is *visible* (not occulted by other parts of the asteroid) and *illuminated* by the Sun is given by:

$$\mathrm{d}L = S(\mu, \mu_0)\varpi \, \mathrm{d}s \tag{1}$$

where:

- L is the observed brightness (light curve).
- $S(\mu, \mu_0)$ and ϖ arise from the scattering law, see Section 1.1.
- ϖ is the *albedo* of the surface patch, which is assumed to be constant across the patch.
- $\mu = \mathbf{E} \cdot \mathbf{n}$ is the projection between the surface unit normal, $\mathbf{n}$ and the unit vector toward the observer (Earth), $\mathbf{E}$.
- $\mu_0 = \mathbf{E_0} \cdot \mathbf{n}$ is the projection between the surface unit normal, $\mathbf{n}$ and the unit vector toward the Sun, $\mathbf{E_0}$.

For the convex case, we may satisfy the conditions of *visibility* and *illumination* by checking that $\mu > 0$ and $\mu_0 > 0$, respectively.

Note also that Equation 1 omits the factor $1/r^2$ and assumes this may be absorbed into the implied normalisation of the light curve intensities, alongside other factors such as the incident light intensity.

Further, the model given by Equation 1 assumes that both the observer and the Sun are sufficiently far away to satisfy the *far-field approximation*. For the Sun, this implies that the incident light upon the asteroid arrives as parallel rays, thus giving a constant incident light direction, $\mathbf{E_0}$ across the asteroid. For the observer, this again implies that the light arrives as parallel rays, which is equivalent to the assumption that the asteroid is observed as an *orthographic projection*.

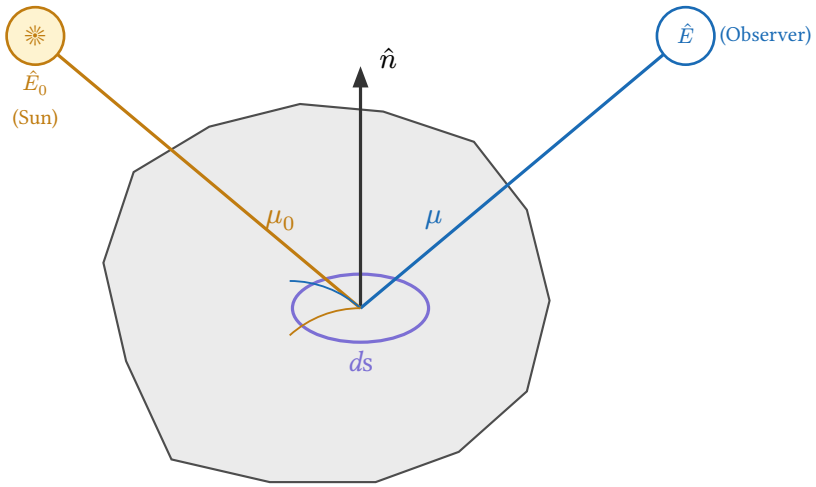


Figure 1: Geometry of light curve contribution.

1.1 Scattering Laws

Common scattering laws include [1]:

$$\begin{aligned} S_{\mathrm{L}} &= \mu\mu_0 && \text{Lambert} \\ S_{\mathrm{LS}} &= \frac{S_{\mathrm{L}}}{\mu + \mu_0} && \text{Lommel-Seeliger} \end{aligned} \tag{2}$$

2 Challenge Setup

We summarise the challenge setup as given by the organisers on the challenge website.

2.1 Geometry

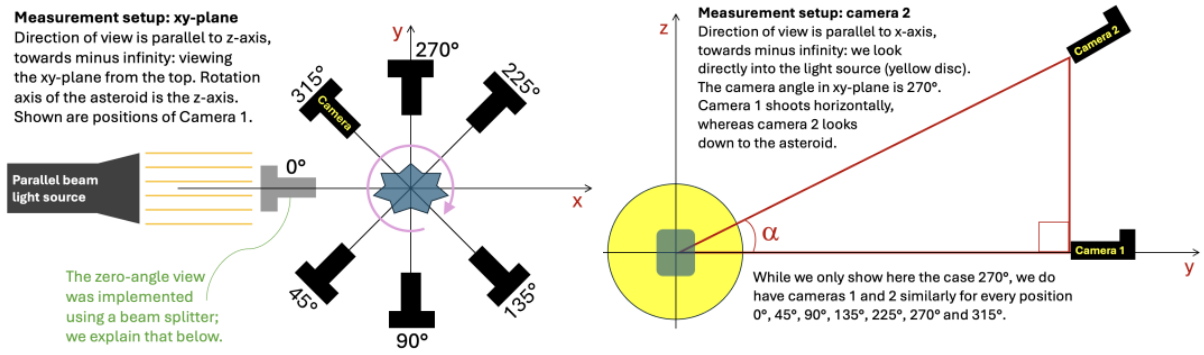


Figure 2: Geometry setup of the asteroid, Sun, and observer (Earth).

So the geometry is set up as:

- Cartesian coordinates (x, y, z)
- Measurement angle θ is the direction of view in the xy-plane, as labelled in the top-view figure. Thus $\theta = 0°$ means the camera looks along $+x$ and is placed on the $-x$ side of the asteroid.
- Elevation angle α is the angle above the xy-plane. Positive α places the camera at positive z.

Thus we have objects:

- Sun at $\mathbf{r}_{\text{sun}} = (-\infty, 0, 0)$ with direction toward the Sun $\hat{\mathbf{E}}_{\text{sun}} = (-1, 0, 0)$. The light travels in the opposite direction, along $+x$.
- Observer at $\mathbf{r}_{\text{obs}} = r_{\text{obs}}(-\cos(\theta)\cos(\alpha), -\sin(\theta)\cos(\alpha), \sin(\alpha))$ with direction toward the observer $\hat{\mathbf{E}}_{\text{obs}} = (-\cos(\theta)\cos(\alpha), -\sin(\theta)\cos(\alpha), \sin(\alpha))$
  - The optical/view direction from the observer to the asteroid is the opposite vector, $(\cos(\theta)\cos(\alpha), \sin(\theta)\cos(\alpha), -\sin(\alpha))$.
  - Where r_{obs} is assumed to be sufficiently large that the asteroid observed approximately as its orthographic projection.

The elevation angles α are given by the organisers on the challenge website and are listed in

Measurement angle, θ	Elevation angle, α
0°	21°
45°	26°
90°	26°
135°	26°
225°	24°
270°	24°
315°	24°

Table 1: Cylinder radius R for each asteroid model.

2.2 Mesh Normalisation

To mitigate the non-uniqueness in the produced shapes which arises from the normalisation of the light curve intensities, we are given information about the *bounding cylinder* of each asteroid model. In particular, we are told that each asteroid is tangent to the planes $z = \pm 1$ and the radius $r = R$, where R is given for each model:

Asteroid	R
Model 1	1.120
Model 2	1.420
Model 3	0.880
Model 4	1.475
Model 5	1.220
Model 6	0.925
Model 7	1.205
Model 8	1.240
Model 9	0.670
Model 10	3.950

Table 2: Cylinder radius R for each asteroid model.

2.2.a Mesh Normalisation Procedure

Let $\mathbf{v}_i = (x_i, y_i, z_i), i = 1, ..., N$ denote the mesh vertices. The procedure normalises the mesh into the challenge cylinder $D(0, R) \times [-1, 1]$, tangent to the planes $z = \pm 1$ with radius $r = R$ (see Table 2).

First, the xy-centre is taken as the midpoint of the xy bounding box,

$$x_c = \frac{1}{2}\Big(\min_i x_i + \max_i x_i\Big), \quad y_c = \frac{1}{2}\Big(\min_i y_i + \max_i y_i\Big), \tag{3}$$

and the vertices are centred in xy:

$$\tilde{x}_i = x_i - x_c, \quad \tilde{y}_i = y_i - y_c. \tag{4}$$

The radial extent after centring is

$$R_{\max} = \max_i \sqrt{\tilde{x}_i^2 + \tilde{y}_i^2}, \tag{5}$$

which fixes the xy-scale factor $s_{\mathrm{xy}} = R/R_{\max}$ so that the normalised mesh has maximum radius exactly R.

Independently, the z-extent is normalised to span $[-1, 1]$. With

$$z_{\min} = \min_i z_i, \quad z_{\max} = \max_i z_i, \quad z_c = \frac{1}{2}(z_{\min} + z_{\max}), \tag{6}$$

the z-scale factor is $s_z = 2/(z_{\max} - z_{\min})$.

The normalised vertices are then

$$\mathbf{v}'_i = \begin{pmatrix} s_{\mathrm{xy}}\tilde{x}_i \\ s_{\mathrm{xy}}\tilde{y}_i \\ s_z(z_i - z_c) \end{pmatrix}. \tag{7}$$

By construction, $\mathbf{v}'_i$ satisfies $\max_i \sqrt{x_{i'}^2 + y_{i'}^2} = R$ and $\min_i z_{i'} = -1$, $\max_i z_{i'} = +1$.

3 Inferring the Observer Position

By inspecting frames from the Blender-simulated videos, we can clear see significant perspective distortion, as shown in Figure 3 and Figure 4. For the Blender simulated data, the left and right edges are clearly not parallel as would be expected for an orthographic view, which suggests that the far-

field observer assumption is severely violated. For the real data, however, the left and right edges are much closer to being parallel, leading us to conclude that the far-field assumption may be valid for the real data.

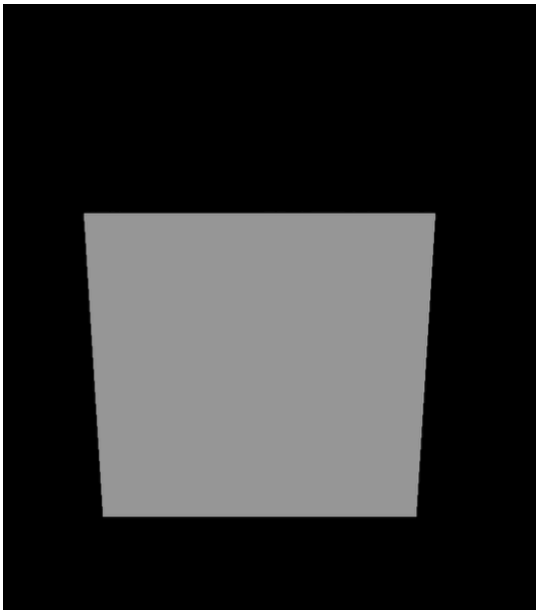


Figure 3: First frame of the simulation video `top0.mp4` for model 2. The camera looks down at the model with an elevation of $\alpha = 21°$ (see Table 1), though the top-face is not visible since it is not illuminated. Image the black area above the visible face to contain the top face of the model. For an orthographic camera, the left and right edges of the visible face would be perfectly parallel. Here they are clearly not, which suggests *significant* perspective distortion.



Figure 4: An equivalent frame from the real video for model 2. The elevation angle is expected to be the same as in the simulation, but we observe a much smaller perspective distortion, which suggests that the camera is much further away. The orthographic assumption may be justified for the real video.

3.1 Camera Position Calibration

The script `calibrate_camera_from_registrations.py` estimates a pinhole camera position and orientation from manually identified registrations. Each registration pairs a known 3D coordinate of a point on the normalised mesh (see Section 2.2) with its pixel position in a video frame. The registration is manual and was carried out using MeshLab and GIMP, with the pairs being recorded in `registrations.json5`.

While producing the registrations was somewhat laborious, the resulting camera calibration seems to be fairly accurate, as can be seen in Figure 5 and Figure 6, which show the registrations overlaid on the corresponding video frames.

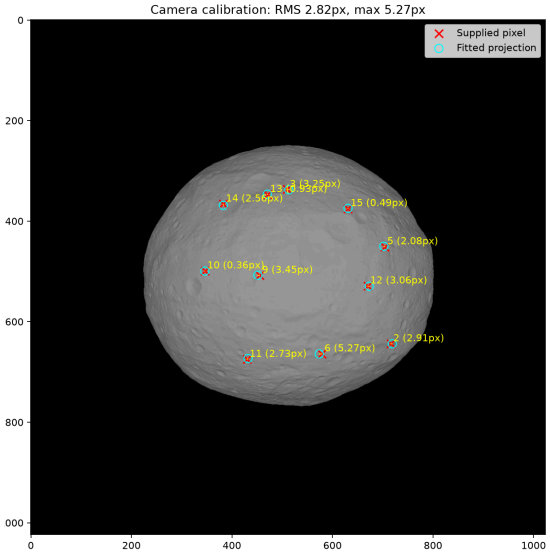


Figure 5: Image showing the registrations for model 1 overlaid the first frame of the simulation video `center0.mp4`.

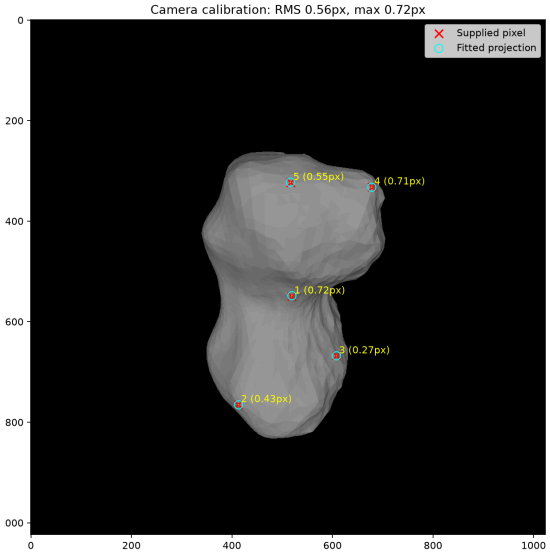


Figure 6: Image showing the registrations for model 3 overlaid the first frame of the simulation video `center0.mp4`.

The obtained camera positions and orientations are summarised in Table 3 alongside the estimated standard deviation of the camera-object distance, r_{obs}.

Asteroid	Dataset	View	Frame	Elevation, α [°],	Azimuth, θ [°]	Distance, r_{obs}
Model 1	`simulated`	`0center`	0	0.29	179.7	3.98 ± 0.10
Model 2	`simulated`	`0bottom`	0	-21.47	180.0	7.42 ± 0.32
Model 2	`simulated`	`0top`	0	21.47	180.0	7.41 ± 0.32
Model 3	`simulated`	`0center`	0	0.13	180.0	4.96 ± 0.29

Table 3: Obtained camera positions and orientations.

Note that only `simulated` data is included, though the inference was also attempted for real videos. These were significantly harder to create registrations for due to the poor quality of the recorded videos. Further, the videos are 'trimmed', which likely invalidates the assumption that the principal point is at the image centre (see Section 3.1.a). Further, these were much closer to the orthographic case, which makes the inference problem ill-conditioned. Rather than try to work around this issue, we instead conclude that the real data is sufficiently far away that the orthographic assumption is valid.

3.1.a Perspective Camera Fitting

Note: This section is a largely LLM generated summary, see Section 7.

For correspondence i, let $\mathbf{P}_i = (X_i, Y_i, Z_i)$ be a known point on the model and let $\mathbf{p}_i = (u_i, v_i)$ be its measured pixel position. If $\mathbf{o}$ is the chosen object centre, the script first centres the world point, $\mathbf{q}_i = \mathbf{P}_i - \mathbf{o}$, and transforms it into camera coordinates:

$$\begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix} = R\mathbf{q}_i + \mathbf{t}. \tag{8}$$

Here $R \in \mathrm{SO}(3)$ is the world-to-camera rotation and $\mathbf{t}$ is the translation after centring the object. The lowercase (x_i, y_i, z_i) are Mitsuba sensor-local coordinates, not the world coordinates (X_i, Y_i, Z_i); world $+z$ remains the up direction. Mitsuba's sensor-local axes are $+x$ to the left, $+y$ up, and $+z$

forward. Raster coordinates u and v increase in the opposite directions to sensor-local x and y, respectively, and a visible point has $z_i > 0$.

The intrinsic model assumes zero skew and square pixels, and therefore uses one focal length f in pixel units. With principal point $\left(c_x, c_y\right)$, perspective division gives the predicted pixel

$$\hat{u}_i = c_x - f\frac{x_i}{z_i}, \quad \hat{v}_i = c_y - f\frac{y_i}{z_i}. \tag{9}$$

By default the principal point is fixed at the image centre, $\left(c_x, c_y\right) = \left(\frac{W-1}{2}, \frac{H-1}{2}\right)$; it can instead be supplied or fitted.

The camera is calibrated by minimising reprojection error:

$$\min_{R,\mathbf{t},f,c_x,c_y} \sum_i \left\| \begin{pmatrix} \hat{u}_i - u_i \\ \hat{v}_i - v_i \end{pmatrix} \right\|_2^2. \tag{10}$$

The principal-point variables are omitted when it is fixed. Rotation is represented by a three-component rotation vector $\boldsymbol{\omega}$, with $R = \exp([\boldsymbol{\omega}]_\times)$. The focal length is parameterised as $f = \exp(\varphi)$ to keep it positive. If $r = \max_i \left\|\mathbf{q}_i\right\|_2$, the forward translation is parameterised as $t_z = r + \exp(\delta)$. Since the forward component of any rotated $\mathbf{q}_i$ is at least $-r$, this guarantees $z_i > 0$ for every supplied point. Thus the fit has seven unknowns—three for rotation, three for translation, and one for focal length—or nine when the principal point is also fitted. At least four or five correspondences, respectively, are needed to provide enough scalar equations, although additional well-spread, non-coplanar points make the solution considerably better constrained.

This is a nonlinear least-squares problem. To reduce the risk of accepting a poor local minimum, the script starts the optimiser from cameras looking at the object from many azimuths and elevations and from four different initial focal lengths, then retains the solution with the smallest residual. The optional robust mode replaces the squared loss on each residual component by a soft-L1 loss, reducing the influence of an incorrectly marked correspondence. After fitting, the singular values of the residual Jacobian are checked to reject a locally rank-deficient calibration in which pose and intrinsics cannot be identified separately.

Finally, the fitted extrinsics are converted into a camera position. If $\mathbf{C}$ denotes that position, then $R(\mathbf{P} - \mathbf{C}) = R(\mathbf{P} - \mathbf{o}) + \mathbf{t}$, and hence

$$\mathbf{C} = \mathbf{o} - R^{\mathsf{T}}\mathbf{t}. \tag{11}$$

In this convention, R maps world-coordinate direction vectors into Mitsuba sensor-local coordinates, so R^{T} is the rotation part of the sensor's `to_world` transform. Equivalently, its `look_at` arguments are

$$\begin{aligned} \text{origin} &= \mathbf{C}, \\ \text{target} &= \mathbf{C} + R^{\mathsf{T}}\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \\ \text{up} &= R^{\mathsf{T}}\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}. \end{aligned} \tag{12}$$

For $\boldsymbol{\Delta} = \mathbf{C} - \mathbf{o}$, the reported distance, elevation, and azimuth are

$$
\begin{aligned}
d &= \|\boldsymbol{\Delta}\|_2, \\
\alpha &= \operatorname{atan2}\Big(\Delta_z, \sqrt{\Delta_x^2 + \Delta_y^2}\Big), \\
\beta &= \operatorname{atan2}\big(\Delta_y, \Delta_x\big).
\end{aligned}
\tag{13}
$$

The reported azimuth β is the bearing of the *camera position*. With the view-direction convention in Section 2.1, the measurement angle is therefore $\theta = \beta - 180°$ modulo $360°$, while the elevation is α. The fitted focal length is converted to fields of view using

$$
\mathrm{FOV}_x = 2\,\mathrm{atan}\left(\frac{W}{2f}\right), \quad \mathrm{FOV}_y = 2\,\mathrm{atan}\left(\frac{H}{2f}\right). \tag{14}
$$

4 Rendering

Several algorithms exist, where it should be noted that we only require a relatively simple ray-tracing algorithm here due to the many simplifications that are possible here as opposed to the general case. For instance, we may assume monochromaticity and ignore atmospheric refraction if we assume the observer is unobstructed by an atmosphere. See also Section 6.1.

A flexible renderer is the Mitsuba framework, which is a research-oriented renderer with inverse rendering capabilities.

Mitsuba-specific considerations are covered below in Section 5

5 Mitsuba

This section is a 'living document' trying to explain some of the considerations and details of the Mitsuba model.

5.1 Variants

In Mitsuba variants describe the data type used to model the light transform, see https://mitsuba.readthedocs.io/en/latest/src/key_topics/variants.html.

We want to model just the *luminance* (single-channel) light levels, and we require auto-diff support (`ad`) and either CUDA or LLVM support, which gives us variants:

- `cuda_ad_mono` (GPU)
- `llvm_ad_mono` (CPU)

5.2 Scene

We need to use a single scene to render everything, which is set up by referencing a single mesh instance at different orientations and translations along the z-axis.

This is set up in `hac.mitsuba.scene`, see Figure 7 for a visualisation of the scene.

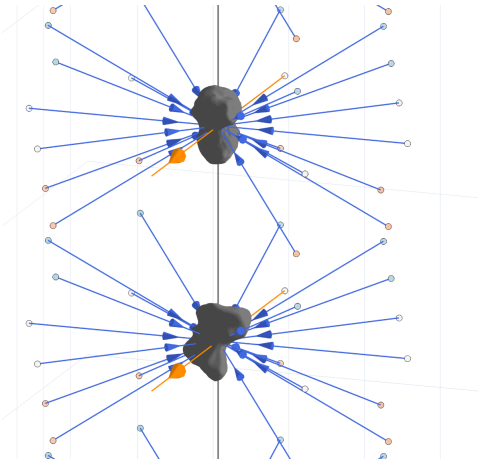


Figure 7: Batch scene setup for Asteroid 3, see `mitsuba_z_stack_viewer.ipynb` for details.

5.3 Forward Model

Within the scene construct described in Section 5.2, we also include parameters of the forward model that are used to render the views that are later turned into light curves. An example of the rendered views prior to reduction into light-curves is shown in Figure 8.

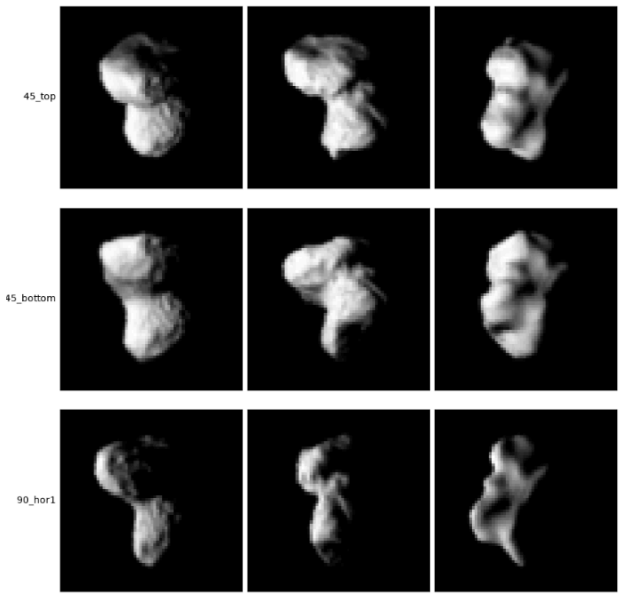


Figure 8: Subset of sensor renderings in a batch scene for asteroid 3 using Mitsuba forward model. Phases are $\phi \in \{0°, 90°, 180°\}$ across, respectively.

5.4 Optimising Parameters of Forward Model

We can use the known geometry and given videos for the first 3 models to optimise the parameters of the forward model.

Unfortunately, the organisers of the challenge have used *perspective rendering for the videos instead of orthographic rendering*. This, combined with the fact that the distance between the asteroid and camera is fairly small[1] and would violate any orthographic assumption, means that we need to produce a reasonable guess of the camera position (alongside the given orientations) in order to produce a reasonable forward model. Intuitively, differences arising from the geometry of the object and aberra-

[1]The distance appears to be approximately 7.3 units for model 2, where 2 is the height of the model as given by the challenge description.

tions stemming from the perspective projection are not easily (at all?) discernable from the perspective of inverse rendering.

Problematically, we believe that the organisers have not used a single, consistent combination of camera positions and object scalings for the videos, which makes it difficult, if not impossible, to produce a forward model that is consistent across all the models. Since the videos are not available for models $4 - 10$, our only hope is that the distances and object scales are *roughly* consistent across the models.

Even if we were able to incorporate the camera position into the optimisation, this is easily achieved with Mitsuba given the constraints described in:

- https://github.com/mitsuba-renderer/mitsuba3/commit/132a3bce69f2ddbff3d434964efb4441a4d37a02
- https://github.com/mitsuba-renderer/mitsuba3/discussions/151#discussioncomment-7097927
- https://github.com/mitsuba-renderer/mitsuba3/issues/1439

See discussion on Teams for more details.

As a best-effort estimate, we can infer the camera position from the simulated videos of the first 3 models. According to the organisers documentation, care has been taken to make the distance and object scaling consistent between the blender simulated video and the real video. Since the videos are not angle synchronised and doing so would be a bit of effort, we only infer the camera positions from the Blender data.

5.5 Update: Orthographic Assumption

The data has been updated to use orthographic rendering. In this case, the silhouette-based `direct_projective` integrator is equivalent to the `direct` integrator, but with extra steps (including invalidation of the cached GPU kernel!), so we can now use the `direct` integrator for the orthographic case, which is much faster.

6 Other

This section contains bits and bobs that don't fit elsewhere in the document.

6.1 Conclusions 2026-06-19

- Standard fully-featured ray tracing algorithms are a poor fit, as we do not need things like:
  - Refraction
  - (Multiple scattering)
  - Polychromaticity

We can get away with simpler algorithms, in particular many papers seem to prefer doing the initial convex checks (visibility + illumination).

7 LLM Disclosure

The code for determining the camera position and orientation from the correspondances was generated with assistance of an LLM. Similarly, the summary of the perspective camera fitting in Section 3.1.a was produced with help from an LLM.

Bibliography

[1] M. Kaasalainen, "Optimization Methods for Asteroid Lightcurve Inversion I. Shape Determination," *Icarus*, vol. 153, no. 1, pp. 24–36, Sep. 2001, doi: 10.1006/icar.2001.6673.